

This section defines the production AWS architecture for an enterprise Agentic AI authorization platform. Every component is justified by its specific role in the authorization chain.

AWS Implementation, Governance & Production Readiness

2.1 Architecture Overview

Ent





1.2 AWS Component Justification Matrix

AWS Service	Role in Architecture	Why This Service
Amazon Verified Permissions (AVP)	Cedar PDP — business and agent authorization	Managed Cedar, no operational burden, native AWS IAM integration, CloudTrail audit
API Gateway (REST/HTTP)	Entry PEP — intercept all external requests	Lambda Authorizer integration, WAF attachment, throttling, caching of auth decisions
Lambda Authorizer	JWT validation + AVP IsAuthorized call + claims normalization trigger	Serverless, per-request, 300-second TTL caching, IAM policy generation
Amazon ECS Fargate	Claims normalization service, MCP PEP gateway, agent runtime	Serverless containers, IAM task roles, VPC networking, no EC2 management
Amazon EKS	OPA sidecar deployment, Kubernetes workloads	OPA Gatekeeper for admission control, IRSA for pod-level IAM
Amazon Bedrock / AgentCore	LLM inference, agent orchestration, tool invocation	Native AWS, Guardrails integration, Bedrock KB for RAG
ElastiCache (Redis)	Claims normalization cache, PDP decision cache	Sub-millisecond cache for normalized claims, reduces PIP lookups >95%
DynamoDB	PIP attribute store, audit decision log, memory metadata	Serverless, single-digit ms, TTL for expiry, per-tenant partition
Amazon OpenSearch	RAG vector search, semantic memory	Vector search with metadata filtering, per-tenant index, fine-grained access control
AWS STS	Token exchange for agent delegation (RFC 8693)	AssumeRoleWithWebIdentity for Entra token exchange, temporary credentials
AWS KMS	Encryption for all data at rest, per-tenant CMK	Envelope encryption, key policy, CloudTrail key usage audit
AWS Secrets Manager	API keys, credentials for downstream systems	Automatic rotation, VPC endpoint, fine-grained IAM access
CloudTrail	Authoritative audit log of all API calls including AVP decisions	Immutable, S3-backed, SNS alerts on policy violations
AWS WAF	L7 protection: rate limiting, IP blocking, OWASP rules	Block known bad actors before reaching auth layer
Step Functions	Human-in-the-loop approval workflow	Wait-for-task-token pattern, Cedar obligation handler
EventBridge	Async event-driven agent workflows, policy change events	Schema registry, rule-based routing, dead-letter queues
AWS Config	Detect policy configuration drift, compliance rules	Custom Config rules for AVP policy store changes

2. Performance Benchmarks & Caching Strategy

Authorization must not become a performance bottleneck. The goal is to add <5ms P99 latency to any request. This requires aggressive caching at multiple layers.

2.1 Performance Targets

Component	P50 Target	P99 Target	Caching Strategy
JWT Signature Validation	<0.5ms	<1ms	JWKS cached in Lambda memory (15 min TTL)
Claims Normalization (cache hit)	<1ms	<3ms	ElastiCache Redis — keyed by token hash
Claims Normalization (cache miss)	<15ms	<30ms	Directory lookup + role mapping + cache write
Cedar AVP IsAuthorized (API)	<3ms	<10ms	Lambda Authorizer IAM policy cache (300s TTL)
OPA Sidecar Evaluation	<1ms	<3ms	Policy bundle in-memory, partial evaluation
Total Authorization Overhead	<5ms	<15ms	All layers combined, warm cache
RAG Pre-filter Construction	<0.5ms	<1ms	Derived from cached canonical claims
Post-retrieval Cedar Evaluation	<2ms/chunk	<5ms/chunk	Batch IsAuthorized for multiple chunks

2.2 Multi-Layer Cache Architecture

REQUEST ■ ▼ Layer 1: Lambda Authorizer IAM Policy Cache ■ • TTL: 300 seconds (max API Gateway cache) ■ • Key: token hash + resource + action ■ • Hit rate: ~70% (repeat API calls) ■ • Latency saved: ~15ms avg per cache hit ■ ▼ Layer 2: Claims Normalization Cache (ElastiCache Redis) ■ • TTL: aligned with JWT expiry (typically 3600s) ■ • Key: SHA256(raw_jwt) ■ • Hit rate: ~95% (token reuse within session) ■ • Latency saved: ~25ms avg per cache hit ■ ▼ Layer 3: PIP Attribute Cache (ElastiCache Redis) ■ • TTL: 300 seconds (short – attributes can change) ■ • Key: userId + attribute_type ■ • Hit rate: ~85% (stable attributes within session) ■ • Latency saved: ~10ms avg per cache hit ■ ▼ Layer 4: OPA Policy Bundle (In-Memory) ■ • TTL: bundle refresh every 30 seconds ■ • Full bundle in OPA memory – no I/O on evaluation ■ • Evaluation latency: <1ms ■ POLICY EVALUATION (all cache layers active) Total overhead: <5ms P99 (warm)

2.3 Cache Invalidation on Policy Change

When policies change, caches must be invalidated to prevent stale authorization decisions. The event-driven invalidation pipeline is:

Policy Change (AVP Console / CI/CD) ■ ▼ CloudTrail → EventBridge Rule (PutPolicy event detected) ■ ▼ Cache Invalidation Lambda • Flush affected Redis keys (by policy scope tag) • Issue API Gateway cache invalidation • Notify OPA bundle server to force refresh ■ ▼ Decision log entry: "POLICY_CHANGED – cache invalidated" Alert: PagerDuty / CloudWatch alarm (if prod change)

3. Policy-as-Code CI/CD Pipeline

Policies are code. They must be version-controlled, peer-reviewed, automatically tested, staged, deployed, and monitored for drift. This is non-negotiable for regulated environments.

3.1 Policy CI/CD Pipeline

Developer writes/modifies Cedar or Rego policy ■ ▼ Git commit to feature branch [GitHub / CodeCommit Repository] ■ ▼ Pull Request opened [Automated PR Checks – GitHub Actions / CodeBuild] • cedar validate (schema check) • cedar test (unit test suite YAML) • OPA conftest (Rego unit tests) • Policy lint (naming conventions, required comments) • Security review flag (policies touching financials auto-flag) • Regression test (shadow evaluation vs production decisions) ■ ▼ PR approved by security team + tech lead [Merge to main] ■ ▼ CI/CD Pipeline triggers [AWS CodePipeline / GitHub Actions] ■ ■■■ Stage 1: Staging Environment ■ • Deploy to staging AVP policy store ■ • Deploy Rego bundle to staging S3 ■ • Run integration tests (100+ scenarios) ■ • Shadow evaluation: compare vs prod (1hr traffic replay) ■ • Approval gate: automated + human sign-off ■ ■■■ Stage 2: Canary Deployment (5% traffic) ■ • Canary AVP policy store ■ • Monitor decision metrics (allow rate, deny rate, errors) ■ • CloudWatch alarm: >2% delta vs baseline → auto-rollback ■ • Duration: 30 minutes minimum ■ ■■■ Stage 3: Production Deployment • Full deployment to production AVP policy store • OPA bundle S3 upload → force refresh all sidecars • CloudTrail event logged: POLICY_DEPLOYED • Rollback trigger: CloudWatch alarm within 15 minutes • Post-deploy: 1-hour monitoring window

3.2 Policy Testing Framework

```
# Cedar Policy Test Suite (YAML format) # File: tests/payment_tool_tests.yaml tests: - name:
"Finance user with MFA can invoke payment tool in business hours" scenario: principal: type:
BankAI::User id: "john.smith" attributes: capabilities: ["can_approve_payment"] mfaVerified:
true businessUnit: "FINANCE" geography: "GB" action: BankAI::Action::"InvokeTool" resource:
type: BankAI::Tool id: "PaymentApprovalTool" context: businessHours: true riskScore: 15
expected: ALLOW - name: "Finance user WITHOUT MFA is denied payment tool" scenario: principal:
type: BankAI::User id: "jane.doe" attributes: capabilities: ["can_approve_payment"]
mfaVerified: false expected: DENY - name: "Out of hours access denied regardless of capability"
scenario: context: businessHours: false expected: DENY - name: "Cross-tenant access always
denied" scenario: principal: attributes: { tenantId: "tenant-a" } resource: attributes: {
tenantId: "tenant-b" } expected: DENY
```

4. Enterprise Case Studies

The following case studies document how industry leaders have implemented externalized authorization, PEP/PDP patterns, and policy-as-code across their organizations:

Capital One — Cedar/AVP for Banking Authorization

Capital One was an early adopter of Amazon Verified Permissions and contributed to Cedar's development. Their architecture uses AVP as the central PDP for all customer-facing banking applications. Key design decisions: capability-based policies abstracted from AD groups, per-microservice PEP enforcement using Lambda Authorizers, and CloudTrail as the authoritative audit record for all authorization decisions. They report <3ms P99 authorization latency at production scale.

Netflix — OPA for Multi-Platform Authorization

Netflix deployed OPA as their primary policy engine across a heterogeneous stack (Java, Python, Node.js). They run OPA as a sidecar in every service pod (Kubernetes), with policy bundles distributed via S3 every 30 seconds. Netflix's key innovation: partial evaluation (OPA's partial eval compiles policies to queries, enabling database-level filtering for authorization). They use this for content authorization and internal service-to-service access control.

Uber — Zanzibar-Inspired ReBAC

Uber built a Zanzibar-inspired authorization system for fine-grained relationship-based access control across their global platform. Key characteristics: relationship tuples stored in globally-consistent storage, authorization decisions computed from relationship graphs, sub-10ms global latency. Uber uses OPA for infrastructure policy and their custom ReBAC system for user-facing authorization.

Goldman Sachs / JPMorgan — FGAC for Financial Data

Major investment banks implement Fine-Grained Access Control (FGAC) using attribute-based models. Their authorization stack includes: ADFS/Entra integration for identity, claims normalization to canonical banking roles (trader, approver, risk manager), Cedar/XACML-style policies for per-instrument data access, and immutable audit trails meeting MiFID II and DORA requirements. Trade-level authorization considers instrument type, trading book, region, and counterparty in a single policy evaluation.

AWS — Cedar Formal Verification

AWS uses Cedar internally for S3 bucket policies, IAM, and Verified Permissions. AWS's contribution to Cedar includes formal verification using the Lean theorem prover — Cedar policies can be mathematically proven to be non-contradictory. This is particularly relevant for banking regulation: authorization policies can be submitted to regulators as formally verified artifacts, not just code.

Intuit — Styra DAS + OPA for Enterprise Scale

Intuit manages OPA across hundreds of services using Styra DAS (Declarative Authorization Service). Their architecture: OPA sidecar per service, policies distributed from Styra DAS central server, decision logs to Elasticsearch, GitOps policy workflow with automated PR reviews. They report 99.99% authorization availability with <2ms P99 sidecar evaluation.

5. Migration Roadmap: Embedded to Centralized Authorization

Migrating from embedded authorization code to a centralized policy engine is a multi-phase program. The following roadmap is sequenced to minimize risk while delivering early value:

Phase 1: Foundation (Weeks 1–6)

Establish the authorization platform foundation: provision AVP policy store, deploy OPA sidecar cluster, implement claims normalization service, establish GitOps policy workflow, configure CloudTrail decision logging. No application changes yet — this phase builds the infrastructure.

- Provision Amazon Verified Permissions (AVP) policy store
- Deploy OPA sidecar cluster on ECS/EKS
- Implement Entra ID / ADFS claims normalization service
- Build canonical claims model and role→capability mapping
- Establish Git repository for policy-as-code
- Configure CloudTrail for AVP decision logging
- Define Cedar entity schema for the domain

Phase 2: Shadow Mode (Weeks 7–12)

Deploy Cedar/OPA in shadow mode alongside existing embedded authorization. Every request is evaluated by BOTH the existing code AND the new policy engine. Compare decisions to identify gaps, mismatches, and edge cases — without changing production behavior.

- Deploy Lambda Authorizer with shadow evaluation mode
- Route all requests through new PEP (pass-through mode initially)
- Compare Cedar decisions vs embedded code decisions
- Log mismatches — resolve each one in policy or canonical claims
- Achieve >99.9% decision parity before proceeding to Phase 3

Phase 3: Canary Enforcement (Weeks 13–18)

Enable policy enforcement for 5–20% of traffic. Monitor closely. Expand gradually as confidence grows. Roll back immediately if mismatch rate exceeds threshold.

- Enable Cedar enforcement for 5% of traffic (canary deployment)
- Monitor authorization metrics in CloudWatch (allow rate, deny rate, latency)
- Expand to 20%, 50%, 80%, 100% over 2-week increments
- Maintain automated rollback trigger: >1% unexpected deny rate
- Complete application code removal of embedded auth checks

Phase 4: Full Production (Weeks 19–24)

Complete migration: 100% policy-engine authorization, all embedded code removed, full audit trail, compliance documentation generated.

- 100% traffic through centralized policy engine
- Remove embedded authorization code from all services

- Complete compliance documentation (NIST, PCI DSS evidence package)
- Policy simulation capability deployed (test before production)
- Policy drift detection active (Config rules)
- Performance optimization complete (cache hit rate >90%)

6. Production Readiness Checklist

The following checklist must be completed before a policy-enforced authorization system is deemed production-ready for regulated enterprise deployment:

Identity & Claims

- JWT signature validation against live JWKS endpoint
- Claims normalization handles both ADFS and Entra ID tokens
- Group GUID resolution with caching (max 5 min TTL)
- Role → capability mapping complete for all business domains
- Nested group flattening tested with 5+ levels
- Large JWT handling: tokens > 4KB use reference token pattern
- Token exchange (RFC 8693) tested for agent delegation

Cedar Policy Design

- Entity schema validated with cedar validate
- Default deny policy is the first policy in every policy set
- Tenant isolation forbid policies present and tested
- All policies reference canonical capabilities (not Entra groups)
- Policy unit tests achieve >95% decision coverage
- Shadow evaluation achieves >99.9% parity with existing system
- Policy versioning and rollback procedure documented and tested

Performance

- P99 authorization latency < 15ms (including all cache layers)
- Claims normalization cache hit rate > 90%
- AVP IsAuthorized P99 < 10ms
- PDP horizontal auto-scaling tested to 10x baseline load
- Cache invalidation tested: stale decisions < 5 minutes after policy change

Security

- All PEPs enforce default-deny: unreachable PDP = deny
- mTLS between all internal authorization components
- Secrets in Secrets Manager (no credentials in environment variables)
- KMS encryption for all data at rest (per-tenant CMK for multi-tenant)
- VPC endpoints for AVP, DynamoDB, S3 (no internet egress for auth traffic)
- GuardDuty enabled and alerting on authorization anomalies
- STRIDE threat model documented and reviewed

Audit & Compliance

- CloudTrail enabled and capturing all AVP policy decisions
- Decision logs include: principal, action, resource, context hash, decision, policy matched
- Audit log retention meets regulatory requirement (PCI DSS: 1 year online, 1 year archive)

- NIST 800-53 control mapping documented
- PCI DSS Requirement 7, 8, 10 evidence package generated
- SOC 2 CC6 control evidence available

Operations

- Policy CI/CD pipeline operational with <15 minute deploy time
- Policy drift detection alert configured (Config rule)
- Emergency policy rollback tested: < 5 minute end-to-end
- On-call runbook for authorization-related incidents
- Monthly policy review meeting scheduled
- Quarterly access review process using Cedar entity audit

Agent-Specific

- Per-step authorization at all 7 agent decision points
- Agent delegation token constrained to minimum required scope
- MCP PEP gateway deployed and tested for all tools
- Human-in-the-loop obligation handler tested for high-risk actions
- RAG pre-retrieval filter tested for tenant isolation
- Memory authorization policies tested for cross-user access prevention
- Output classification filter deployed and tested for PII leakage

7. Final Decision Framework

The decision framework summarizes when to use Cedar, when to use OPA/Rego, and when to use both. This is the definitive guidance for enterprise architects:

Use Case	Use Cedar (AVP)	Use OPA/Rego	Use Both (Hybrid)
Application authorization	✓ Primary choice		If cross-cloud needed
Agent permissions	✓ Best fit — typed, auditable		
Tool invocation control	✓ Per-tool entity model		
Kubernetes admission		✓ First-class support	
Terraform/IaC policy		✓ Conftest standard	
Infrastructure security		✓ Universal	
AWS-native deployment	✓ No operational burden		OPA for K8s layer
Multi-cloud deployment		✓ Cloud-agnostic	
Formal verification needed	✓ Provably correct		
Large enterprise (>100 services)			✓ Cedar for business, OPA for infra
RAG authorization	✓ Document entity model		
Memory protection	✓ User scope model		
Service mesh policy		✓ Envoy/Istio native	
Edge/WASM deployment		✓ WASM compilation	
Regulatory banking compliance	✓ CloudTrail native audit		OPA for infra compliance

✓ BEST PRACTICE

Final Recommendation: For the described environment (AWS, Entra ID, Agentic AI, banking regulation), deploy Amazon Verified Permissions (Cedar) as the primary authorization engine for all application, agent, tool, RAG, and memory authorization decisions. Deploy OPA/Rego for Kubernetes admission control (EKS), Terraform policy gates, and infrastructure compliance. Use a shared claims normalization service to provide canonical claims to both engines. This hybrid architecture is the most mature, auditable, and operationally sound choice for regulated financial services.